WARP-K: Watt-Aware Reinforcement Policy for Kernels

Gabriel Bo [1] **Yohannes Aklilu** [1]

ABSTRACT

AI-driven datacenter electricity demand is projected to more than double by 2030, making energy a first-class systems constraint alongside compute. However, automated GPU kernel generation pipelines—including LLM synthesis and agentic optimization on benchmarks such as KernelBench—still optimize mainly for correctness and throughput, even though kernels with similar runtime can differ substantially in joules due to instruction mix, memory traffic, and utilization. We introduce WARP-K (Watt-Aware Reinforcement Policy for Kernels), a multi-objective RL framework that trains Qwen2.5-Coder-7B with GRPO and LoRA to generate energy-efficient kernels using hardware-grounded rewards (Qwen Team, 2025). Our reward incorporates joules, average power, and intelligence-per-watt (IPW), computed from concurrent NVML telemetry collected on H100 GPUs (NVIDIA, 2023b). The telemetry stack is implemented as a Rust gRPC service streaming 11 GPU metrics at 50 ms intervals, enabling trapezoidal energy integration and phase-level attribution directly inside the training loop rather than as post hoc analysis (NVIDIA, 2023b;c). As an exploratory extension, WARP-K also includes hierarchical runtime control over deployment knobs (power caps, DVFS profiles, and concurrency) under latency SLA constraints. Across 80 KernelBench tasks, four difficulty levels, and three seeds, energy-aware GRPO reduces energy by 24% versus throughput-only RL at similar correctness (65.4% vs. 66.7%). A telemetry-aware variant lowers energy by 35% (3.76 J vs. 5.82 J) while achieving the best correctness (67.5%). Under matched-runtime comparisons, it reaches 37% lower median joules with under 3% runtime degradation. Ablations show these gains come from explicit energy-aware reward design, demonstrating that hardware-informed kernel generation is a practical path to more energy-efficient AI infrastructure.

1. Introduction

The rapid scaling of generative AI has made energy a first-class systems constraint. As large language models expand in training and deployment, datacenter growth is increasingly limited not only by compute availability, but by electricity, power delivery, and infrastructure cost (International Energy Agency, 2025; Shehabi et al., 2024). In this regime, reducing energy consumption while maintaining performance does more than improve technical efficiency: it directly increases the effective capacity of frontier AI labs by allowing the same infrastructure budget to support more training, more inference, and higher overall throughput.

At the same time, language models are increasingly being used to generate the low-level software that powers AI infrastructure itself. In particular, recent systems use LLMs and agentic optimization to generate GPU kernels, the performance-critical routines that govern memory movement, hardware utilization, and end-to-end throughput in training and inference pipelines (Ouyang et al., 2025; Baronio et al., 2025; Liao et al., 2025; Dai et al., 2026). This makes kernel generation an unusually important optimization target since the code produced by one model can directly shape the efficiency of future AI workloads at scale.

Yet current kernel-generation systems and benchmarks are optimized primarily for correctness and speed. KernelBench, for example, evaluates whether models generate kernels that are both correct and faster than PyTorch baselines, but does not optimize for energy efficiency under power constraints (Ouyang et al., 2025). More broadly, prior work in efficient ML and energy-aware systems argues that energy should be treated as a first-class metric (Strubell et al., 2019; Schwartz et al., 2020; You et al., 2022; MLCommons, 2025), but these efforts do not train kernel-generation policies directly on telemetry-derived hardware energy signals.

This gap motivates our work. We observe that kernels with nearly identical runtime can still exhibit substantially different energy consumption due to differences in instruction mix, memory traffic, occupancy, and power behavior. Prior work on GPU profiling and DVFS further shows that energy-performance tradeoffs are workload-dependent and cannot be recovered reliably from runtime alone (Sen et al., 2018; Ma et al., 2017; Ali et al., 2023). These observations suggest

[1]Stanford University. Correspondence to: Gabriel Bo <gabebo@stanford.edu>, Yohannes Aklilu <yaklilu@stanford.edu>.

that energy-efficient kernel generation requires a training objective grounded in direct telemetry rather than proxy assumptions.

We present **WARP-K** (**W**att-**A**ware **R**einforcement **P**olicy for **K**ernels), a framework for training language models to generate energy-efficient GPU kernels through multi-objective reinforcement learning grounded in hardware telemetry. WARP-K makes two core contributions in the main paper. First, we extend Group Relative Policy Optimization (GRPO) with reward terms for joules, average power, and intelligence-per-watt (IPW), enabling direct optimization of energy-aware code generation. Second, we build a telemetry-integrated training pipeline in which a Rust gRPC service streams NVML metrics from H100 GPUs at 50 ms intervals, enabling online energy integration and phase-level attribution inside the training loop. We then show empirically that performance and energy are separable objectives in kernel generation: across 80 KernelBench tasks, our telemetry-aware model reduces energy by 35% relative to throughput-only RL while maintaining comparable or better correctness, and under matched-runtime comparisons achieves a 37% energy advantage. Runtime-control policies (M4/M5) are treated as exploratory extensions and are reported primarily in Appendix D. Together, these results show that telemetry-aware reinforcement learning can shift the performance–energy frontier for automated kernel generation.

2. Related Work

LLM-based kernel generation. KernelBench established LLM-based GPU kernel generation, and later systems (Kevin, KernelEvolve, CUDA Agent) extend iterative optimization toward deployment; however, this line still primarily optimizes correctness and throughput rather than telemetry-grounded energy (Ouyang et al., 2025; Baronio et al., 2025; Liao et al., 2025; Dai et al., 2026).

RL for systems and hardware optimization. RL has been applied from sequential systems control to agentic LLM feedback loops, including chip placement and algorithm discovery; Zeus is the closest energy-aware systems baseline, but it tunes runtime controls around fixed kernels, whereas WARP-K applies energy-aware RL at kernel-generation time (Kaelbling et al., 1996; François-Lavet et al., 2018; Zhang et al., 2025; Mirhoseini et al., 2021; Google DeepMind, 2024; Mankowitz et al., 2023; You et al., 2022).

Energy measurement and efficiency-aware evaluation. Prior profiling work shows GPU energy measurement is noisy and workload dependent, while Green-AI and benchmark efforts formalize efficiency-aware evaluation; WARP-K operationalizes this by training directly on telemetry-derived rewards with IPW-style terms (NVIDIA, 2023b; 2024; Sen et al., 2018; Ma et al., 2017; Ali et al., 2023; Strubell et al., 2019; Schwartz et al., 2020; MLCommons, 2025; Salesforce, 2025; Chung et al., 2025; Saad-Falcon et al., 2025).

3. Preliminaries

This section presents WARP-K's learning framework: a multi-objective formulation for kernel generation, GRPO for code synthesis, and PPO for runtime control.

3.1. Problem Formulation

We formulate energy-aware kernel generation as multi-objective optimization over code. Given task τ (a PyTorch reference and input signature), the model generates CUDA code c that is simultaneously *correct*, *fast*, and *energy-efficient*.

We model this process as a Markov Decision Process (MDP). At step t, the state s_t consists of the prompt context, including the task specification, any previous generations, and execution feedback such as compilation errors or measured performance. The action a_t is a code completion sampled from a policy π_θ. After the generated code is compiled and executed on GPU hardware, the environment returns a scalar reward $r(s_t, a_t)$ derived from compilation status, correctness, runtime, energy, and power measurements.

Our kernel-generation policy π_θ is parameterized by `Qwen2.5-Coder-7B-Instruct` (Qwen Team, 2025), adapted with LoRA (Hu et al., 2022). We apply LoRA to the query, key, value, and output projection matrices with rank $r = 64$ and scaling factor $\alpha = 16$. This design provides a parameter-efficient way to specialize the base model to the kernel-generation domain while retaining the flexibility needed for reinforcement learning.

3.2. Group Relative Policy Optimization (GRPO)

For kernel generation, WARP-K uses Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which removes the need for a learned value function by normalizing rewards within prompt-level candidate groups. This is well suited to code generation, where rewards are sparse and expensive to evaluate. Figure 1 summarizes this grouped rollout.

For each task τ_i, we sample a group of $k = 8$ candidate kernels,

$$\{c_1, \ldots, c_8\} \sim \pi_\theta(\cdot \mid \tau_i). \tag{1}$$

Each candidate is compiled, executed, and measured to obtain a scalar reward r_j. GRPO computes a group-relative advantage for candidate j as

$$\hat{A}_j = \frac{r_j - \mu_{\mathbf{r}}}{\sigma_{\mathbf{r}} + \epsilon}, \tag{2}$$

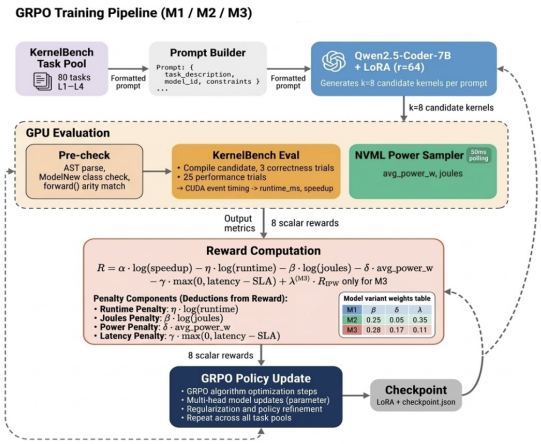


Figure 1. GRPO training pipeline for grouped kernel generation, execution-based evaluation, and policy updates.

where $\mu_{\mathbf{r}}$ and $\sigma_{\mathbf{r}}$ are the mean and standard deviation of the rewards within the sampled group.

The policy is updated using a clipped surrogate objective with an explicit KL penalty against a reference policy:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}\Big[\min\Big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\Big)\Big] + \beta_{\text{KL}}\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}), \tag{3}$$

where

$$\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} \tag{4}$$

is the importance ratio and $\beta_{\text{KL}} = 0.04$.

We adopt Kevin-32B's grouped-rollout strategy (Baronio et al., 2025) with trajectory filtering: from each group of 8 candidates, we retain the top $k = 4$ by correctness and speed signals, and allow up to two compiler-feedback repair rounds before final scoring.

GRPO is used for models M1–M3. Across these stages, the optimization algorithm remains fixed; only the reward function changes. This isolates the effect of reward design and enables controlled comparison between throughput-only and energy-aware objectives: M1 isolates speed-only RL, M2 tests direct energy terms, and M3 tests whether IPW-style efficiency adds value beyond joule minimization.

3.3. Proximal Policy Optimization for Runtime Control

For runtime control in M4 and M5, we use Proximal Policy Optimization (PPO) (Schulman et al., 2017) with a separate actor-critic architecture. Unlike kernel generation, which operates over token sequences, runtime control acts over a compact hardware-configuration space and is naturally modeled as a standard control problem.

The runtime state includes queue depth, batch size, current power cap, DVFS profile, concurrency level, and recent latency statistics such as $p95$ response times. The action space is discrete and consists of runtime configurations over three control axes:

- **Power caps:** $\{350, 450, 550, 650, 700\}$ W
- **DVFS profiles:** {`efficiency`, `balanced`, `performance`}
- **Concurrency levels:** $\{1, 2, 4, 8\}$

The actor-critic network is a two-layer MLP with hidden dimension 128. We train it with learning rate 3×10^{-4}, discount factor $\gamma = 0.99$, GAE parameter $\lambda = 0.95$, PPO clip ratio 0.2, and entropy coefficient 0.01. Updates occur every 5 episodes over a horizon of 10 steps. This controller learns policies that trade off latency, throughput, and energy under deployment constraints.

3.4. Hierarchical RL (M5)

Model M5 combines kernel generation and runtime control in a hierarchical reinforcement learning setting. Intuitively, M1–M3 optimize what code is generated, while M4 optimizes how that code is executed; M5 jointly optimizes both.

Training alternates between the two levels of control: we perform two rounds of one epoch of kernel GRPO followed by 10 episodes of runtime PPO, then complete training with 5 episodes of joint fine-tuning. A lightweight hierarchy controller, implemented as a two-layer MLP with hidden dimension 64, selects among kernel families (*balanced*, *prefill-optimized*, *decode-optimized*, *throughput-optimized*, and *energy-optimized*), while the runtime controller selects hardware configurations such as power cap and concurrency.

4. Datasets, Profiling, and Telemetry

4.1. Dataset: KernelBench

We use KernelBench (Ouyang et al., 2025) as the primary source of kernel-generation tasks, spanning four levels of difficulty:

- **Level 1 (L1):** single-operator primitives, such as GEMM, convolution, activations, and reductions
- **Level 2 (L2):** fused operator chains, such as convolution–batchnorm–ReLU
- **Level 3 (L3):** architecture blocks, such as ViT encoder layers or LSTM cells
- **Level 4 (L4):** full-model inference components, such as small LLM modules or detection heads

We train on the full KernelBench corpus and evaluate the main paper results on a fixed 80-task test-only suite spanning all four difficulty levels. No tasks from this 80-task suite are used for gradient updates; model checkpoints are selected using training-time validation signals on non-evaluation tasks. Each evaluation task provides a reference PyTorch `Model` class and `forward()` signature; the model must generate an equivalent `ModelNew` implemented with custom CUDA kernels compiled through `torch.utils.cpp_extension.load_inline`. Results are reported across three seeds (11, 22, 33).

4.2. Evaluation Protocol

Each generated candidate kernel is evaluated through a three-stage pipeline: **1. Structural pre-check.** We first verify that the candidate parses successfully, defines a `ModelNew` class, and matches the expected `forward()` arity.

2. Correctness evaluation. We execute the candidate on randomized inputs for 3 trials and compare outputs against the reference implementation within a task-specific tolerance.

3. Performance and telemetry evaluation. For candidates that pass correctness, we run 25 timed trials with warmup using CUDA events to obtain `runtime_ms` and speedup relative to the PyTorch baseline.

During the performance trials, a concurrent telemetry thread samples power at 50 ms intervals, yielding a time series of (t_i, P_i) pairs used for downstream energy integration.

4.3. Telemetry Pipeline

Reliable energy measurement is central to WARP-K because the reward depends directly on hardware behavior. We therefore use a tiered telemetry stack with progressively simpler backends based on hardware/software availability. Figure 2 illustrates the end-to-end data path used during training and evaluation.

Tier 1: H100 Rust gRPC sampler (primary). The primary backend is a standalone Rust service (`tools/h100-energy-sampler/`) built with `tonic` and `nvml_wrapper`. It exposes `H100TelemetryService::StreamSamples`, emitting `TelemetrySample` messages at configurable intervals (default 50 ms) with 11 per-GPU metrics (including power, cumulative energy, utilization, clocks, memory, PCIe throughput, and throttle state). Where possible, power is derived from energy-counter deltas rather than instantaneous readings.

Tier 2: IPW stack. When the Intelligence-Per-Watt profiling stack is available, we use `EnergyMonitorCollector` and

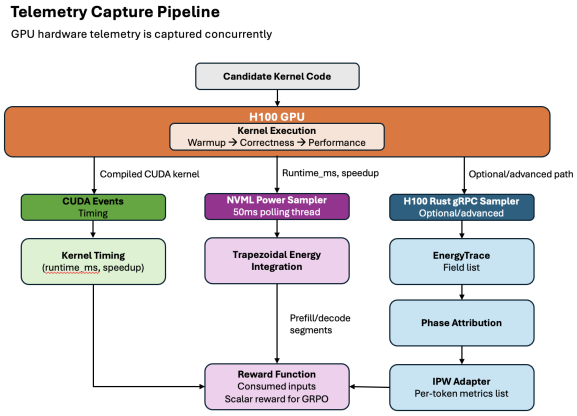


Figure 2. Telemetry pipeline architecture and data flow used for online energy-aware training and evaluation.

`TelemetrySession` to obtain per-query joules, token-level breakdowns, and throughput metrics.

Tier 3: Python NVML fallback. If the Rust service and IPW stack are unavailable, a Python daemon thread polls `pynvml.nvmlDeviceGetPowerUsage()` every 50 ms and records timestamped wattage samples.

Tier 4: Synthetic traces. For development and testing without GPU access, we provide a synthetic-trace backend that mimics the structure of real telemetry.

This layered design allows the same training and evaluation code to run across heterogeneous environments while preserving a consistent telemetry interface.

4.4. Energy Integration

Given a sequence of power samples

$$\{(t_0, P_0), (t_1, P_1), \ldots, (t_n, P_n)\}, \tag{5}$$

we compute total energy by trapezoidal integration, which provides a numerically stable estimate of total joules under time-varying GPU power:

$$E = \sum_{i=1}^{n} \frac{1}{2}(P_{i-1} + P_i)\,\Delta t_i, \qquad \Delta t_i = \max(0, t_i - t_{i-1}). \tag{6}$$

Average power is then computed as

$$\bar{P} = \frac{E}{\sum_i \Delta t_i}. \tag{7}$$

These two quantities, `total_joules` and `avg_power_w`, are the primary telemetry-derived signals used in the reward function. Additional implementation details—including polling-interval justification, edge-case handling

for `max(0, t_i - t_i-1)`, dropped-sample handling, and integrator pseudocode—are provided in Appendix B.

4.5. Phase Attribution

For inference-like workloads, energy use is not uniform across execution, so we partition each telemetry trace into *prefill* and *decode* using the time-to-first-token (TTFT) boundary when available, or 30% of total duration as a fallback. This yields phase-specific estimates such as:

- `prefill_energy_per_input_token`
- `decode_energy_per_output_token`

These phase-level metrics feed into the IPW adapter, which additionally computes

$$\mathrm{TPS} = \frac{\text{input tokens} + \text{output tokens}}{\text{duration}}, \tag{8}$$

$$\mathrm{IPW} = \frac{\text{throughput}}{\bar{P}}, \qquad \mathrm{IPJ} = \frac{\text{throughput}}{E}. \tag{9}$$

This decomposition is particularly useful for runtime-control experiments, where the most energy-sensitive operating point may differ between prefill and decode. Exact TTFT extraction logic, the 30% fallback rule, token-normalized phase-metric formulas, and extended IPJ details are deferred to Appendix B.

4.6. Reward Functions

The reward function is the point at which telemetry enters the learning loop. All WARP-K stages use a common composable reward template, with different coefficients depending on the experiment.

GRPO Multi-Metric Reward (M1–M3). For a candidate that compiles and passes correctness, we define the reward as

$$\begin{aligned} R = {} & \alpha \ln(\text{speedup}) - \eta \ln(\text{runtime}_{\text{ms}}) \\ & - \beta \ln(\text{joules}) - \delta\, \text{avg_power}_{\text{W}} \\ & - \gamma \max(0, \ell_{p95} - \ell_{\text{SLA}}). \end{aligned} \tag{10}$$

Each term is included for a specific systems reason: $\alpha \ln(\text{speedup})$ preserves the standard kernel-generation objective relative to baseline throughput; $-\eta \ln(\text{runtime}_{\text{ms}})$ prevents the policy from converging to low-power but impractically slow kernels; $-\beta \ln(\text{joules})$ directly optimizes total energy, our primary systems objective; $-\delta\, \text{avg_power}_{\text{W}}$ discourages brief but power-hungry kernels that stress datacenter power envelopes; and the SLA term prevents efficiency gains that violate latency constraints.

Stage	α	β	γ	δ	η	λ	Objective
M1	1.0	0.0	0.25	0.0	0.10	0.0	Speed only
M2	1.0	0.5	0.25	0.01	0.10	0.0	+ Energy
M3	1.0	0.5	0.25	0.01	0.10	0.0–0.5	+ IPW blend

Table 1. Reward coefficients for GRPO stages.

We use logarithms for speedup, runtime, and joules to compress scale differences and reduce sensitivity to outliers.

Candidates that fail to compile receive $R = -1.0$, while candidates that compile but fail correctness receive $R = -0.5$. Out-of-memory events incur an additional -0.5 penalty. These explicit penalties are important because code generation has discontinuous failure modes; without hard penalties, the reward landscape does not cleanly separate valid from invalid candidates, which destabilizes policy optimization.

The staged M1–M3 design is intentional: M1 isolates the effect of speed-only RL, M2 tests whether direct energy terms change the learned policy, and M3 tests whether a broader efficiency notion (IPW) adds value beyond plain joule minimization.

The model stages differ only in their reward coefficients:

For M3, we add an IPW blend term:

$$R_{\text{total}} = R_{\text{GRPO}} + \lambda R_{\text{IPW}}, \tag{11}$$

where

$$\begin{aligned} R_{\text{IPW}} = {} & \alpha \ln(\text{speedup}) - \beta_{\text{IPW}} \ln(\text{joules}) \\ & - \gamma_{\text{IPW}} \max(0, \ell_{p95} - \ell_{\text{SLA}}), \end{aligned} \tag{12}$$

with $\beta_{\text{IPW}} = 0.5$, $\gamma_{\text{IPW}} = 0.25$, and $\lambda \in \{0.0, 0.1, 0.25, 0.5\}$ swept as a hyperparameter.

HRL Reward (M4–M5). For runtime control, the PPO policy receives

$$\begin{aligned} R_{\text{HRL}} = {} & 0.35\,\text{TPS} + 0.25\,\text{APJ} \\ & + 0.20\,\text{APW} + 0.05\,\text{stability} \\ & - 0.15\,\text{latency_violation}, \end{aligned} \tag{13}$$

where APJ and APW denote accuracy-per-joule and accuracy-per-watt, `stability` measures reward variance over recent episodes, and

$$\begin{aligned} \text{latency_violation} = {} & \max(0, \text{TTFT}_{p95} - \text{SLA}) \\ & + \max(0, \text{E2E}_{p95} - \text{SLA}). \end{aligned} \tag{14}$$

This reward differs from the kernel-generation reward by design: runtime control optimizes serving behavior rather than code synthesis, so it emphasizes throughput, efficiency, stability, and SLA compliance. We include `stability` to favor policies that remain reliable across episodes and load patterns.

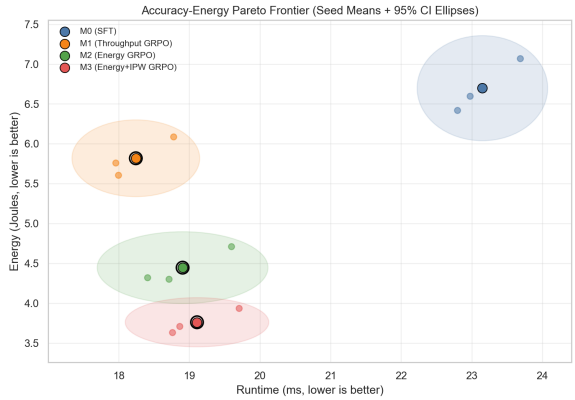


Figure 1. Pareto frontier showing M0–M3 in runtime–energy space with confidence ellipses.

5. Results

Main-text results focus on kernel generation. Runtime control, ablations, and cross-hardware transfer are deferred to Appendix D. We compare M1 (throughput-only GRPO), M2 (energy-aware GRPO), and M3 (IPW-blended energy-aware GRPO); M4 and M5 are exploratory runtime-control extensions. All headline results are reported on a fixed 80-task test suite over three seeds, with checkpoints selected using validation signals on disjoint non-evaluation tasks. For fairness, M1-M3 share the same base model, rollout and repair budgets, and evaluation protocol; reward design is the primary difference. M0 is the supervised baseline, and M1 is the throughput-oriented RL baseline. All kernel experiments run on NVIDIA H100 SXM (80 GB, 700 W TDP) (NVIDIA, 2023a) with telemetry sampled every 50 ms. We report compile rate, correctness, pass@k, runtime, energy, average power, speedup over PyTorch, and aggregate reward.

5.1. Model Progression: From SFT to Energy-Aware RL

Table 1 is the anchor result table. It establishes the global progression from M0 to M3: throughput-only RL (M1) improves correctness and speed, while telemetry-aware RL (M2/M3) shifts the model toward a better efficiency frontier. M5 is the fastest overall stage, but among kernel-generation stages the core objective is best achieved by M3, which gives the strongest balance of correctness, energy, power, and reward.

Figure 1 is the central visual claim of the paper. Across stages, the frontier moves toward lower runtime and lower energy, with M3 dominating the kernel-generation region. This is not just higher reward; it is a shift in the feasible tradeoff surface, consistent with the table-level gains in Table 1.

The per-task runtime–energy cloud in Figure 2 adds an important complement: for comparable runtime bands, M3 trends lower in energy. This shows the effect is not only at model-level averages but across task-level behavior, and it foreshadows the matched-runtime result.

Figure 3 explains why the staged pipeline matters: M0→M1 contributes the largest jump, then M2 and M3 add additional efficiency-oriented gains. This supports the M0→M1→M2→M3 design rather than treating later stages as redundant.

Finally, Figure 4 shows difficulty stratification: all models perform best on easier levels and degrade on harder tiers, but telemetry-aware policies remain competitive while improving efficiency. This motivates the deeper difficulty-level analyses in Appendix D (e.g., Figure 6(b) and Figure 5).

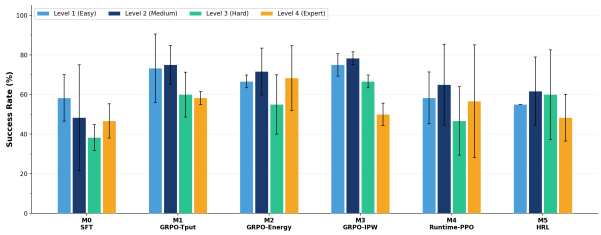


Figure 4. Per-level (L1–L4) correctness for M0–M5.

5.2. Matched-Runtime Energy Advantage

The strongest evidence that runtime and energy are distinct optimization objectives comes from the matched-runtime experiment. We identify task-seed pairs in which M3 and M1 achieve runtimes within 3% of one another, then compare the resulting energy consumption.

Figure 7 and Table 2 provide the strongest controlled matched-runtime evidence in the paper. Even with runtime nearly fixed, M3 still reduces energy by about 37% versus M1 with very strong statistical support. This directly rules out the simpler explanation that energy gains come only from faster execution; the policy is learning different kernel behaviors, not merely shorter runtimes.

5.3. Difficulty-Stratified Analysis and Per-Task Energy Deltas

Figure 5 shows that gains are broad-based rather than driven by a few outliers, with the largest absolute savings concentrated on harder tasks. Combined with Figure 2, this supports the claim that harder, data-movement-heavy workloads expose greater room for speed-equivalent but energy-different implementations. A concrete, engineer-readable example is provided in Appendix B.3 (Task L1-40, LayerNorm), which presents two real generated kernels

Table 1. Main model comparison. All metrics are averaged over 80 tasks × 3 seeds.

Model	Stage	Correctness	Pass@k	Runtime (ms)	Joules	Avg Power (W)	Speedup	Reward
M0	SFT Baseline	0.479	0.675	23.15	6.70	250.7	1.19x	-0.02
M1	Throughput GRPO	0.667	0.792	18.24	5.82	276.6	1.92x	8.00
M2	Energy GRPO	0.654	0.804	18.90	4.45	205.1	1.67x	9.17
M3	IPW Blend	0.675	0.792	19.11	3.76	174.8	1.61x	10.27
M4	Runtime PPO	0.567	0.721	17.34	4.44	222.8	1.63x	8.32
M5	Hierarchical	0.563	0.750	15.76	3.77	207.2	1.72x	9.97

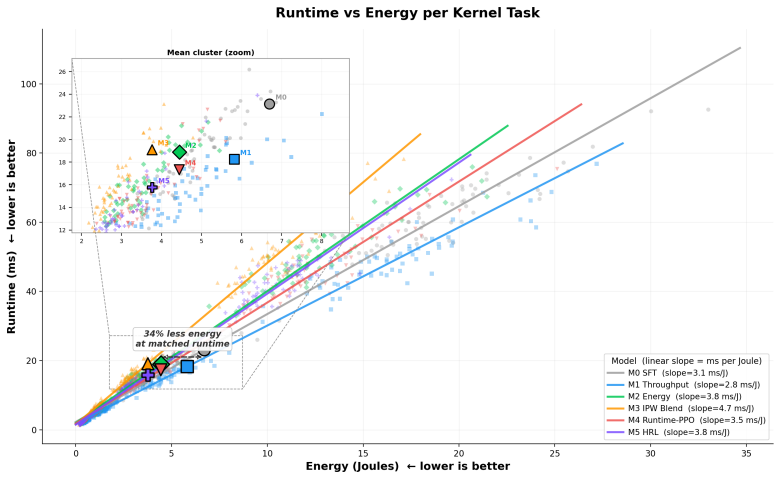


Figure 2. Per-task runtime–energy scatter comparing model stages M0–M3 over the KernelBench evaluation (80 tasks, four difficulty levels, three random seeds) on H100 GPUs with NVML telemetry sampled at 50 ms. Points show that at similar runtime bands, telemetry-aware policies (especially M3) consistently occupy lower-energy regions than throughput-only RL baselines. The widening spread at longer runtimes indicates harder workloads expose larger energy differences even when latency is comparable.

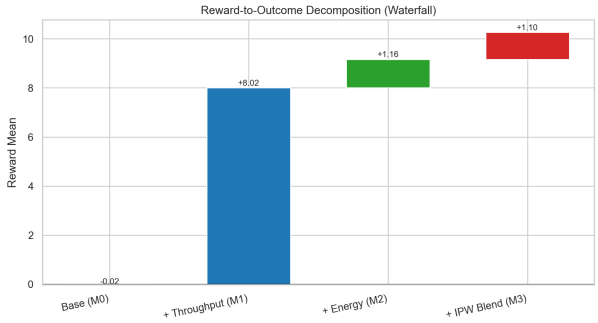


Figure 3. Marginal reward contribution of each stage (M0→M3).

(throughput-optimized vs. energy-aware) and explains why they achieve similar correctness and runtime but materially different power/energy behavior.

In Appendix D, we report additional experimental results for runtime control (Table 6, Figure 18), ablation analysis (Table 7, Figure 15(b)), and cross-hardware transfer (Table 8, Figure 17(a)). These results complete the core argument from the introduction: code-generation objectives and runtime policies must both be energy-aware, and their benefits depend on hardware-grounded telemetry rather than speed-only proxies.

Table 2. Matched-runtime energy comparison.

Metric	Value
Median Δ Joules (%)	-37.3
IQR (%)	[33.5, 42.1]
Wilcoxon p	$< 10^{-6}$
Effect size (d)	5.40

5.4. Summary

Taken together, the results support the main WARP-K hypothesis that energy-aware reinforcement learning changes what kernels are generated, not just how they are scored. Table 1 and Figure 1 show a frontier shift: throughput-only RL (M1) improves correctness and runtime relative to M0, but it also raises average power from 250.7 W to 276.6 W, indicating that speed-only optimization can push the policy toward more hardware-hungry kernels. In contrast, telemetry-aware rewards in M2 and M3 substantially reduce joules (4.45 J and 3.76 J vs. 5.82 J for M1) and average power (205.1 W

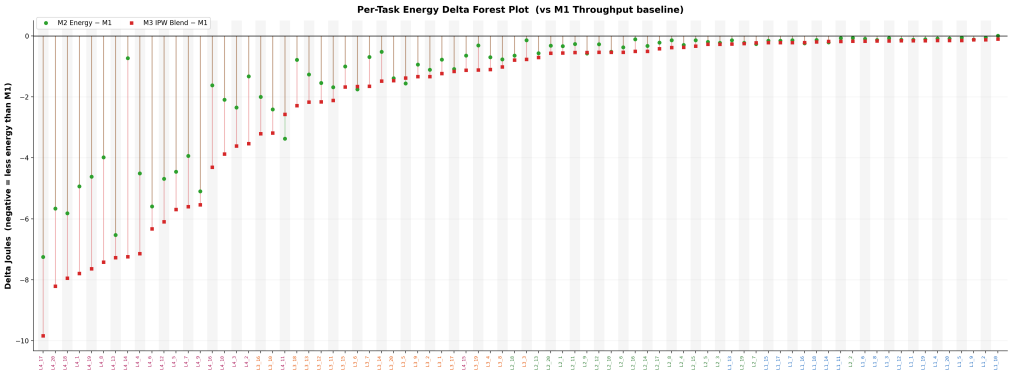


Figure 5. Per-task energy-delta forest plot for telemetry-aware stages versus throughput-only RL (M2–M1 and M3–M1), sorted by absolute effect size across the KernelBench evaluation set (80 tasks, three seeds). Most tasks show negative deltas, indicating broad energy savings rather than gains driven by a few outliers. Larger absolute improvements on harder tasks are consistent with higher memory-traffic sensitivity, where energy-aware optimization has more room to improve.

and 174.8 W vs. 276.6 W), while preserving competitive correctness. Among kernel-generation stages, M3 provides the strongest combined profile and therefore best reflects the paper's objective of efficient kernels at competitive performance.

The matched-runtime analysis is the clearest evidence that energy and runtime are separable objectives. When M3 and M1 are constrained to within 3% runtime, M3 still delivers a median -37.3% energy reduction (Table 2) with strong statistical support (Wilcoxon $p < 10^{-6}$, effect size $d = 5.40$). This rules out the interpretation that gains come only from faster execution. Combined with the per-task and distributional views (Figures 2 and 5), the evidence indicates that WARP-K learns different hardware behavior—instruction mix, memory traffic, occupancy, and power draw—with the largest improvements on harder, data-movement-heavy tasks that are most representative of practical transformer-serving bottlenecks.

AAppendix analyses further support mechanism and robustness. Ablations (Table 7, Figure 15(b)) show that removing any reward term worsens energy, with joules rising from 3.21 J to 3.68–4.61 J, indicating that the gain comes from the combined objective rather than any single component. Calibration and seed-stability results (Figures 12(b) and 10(a)) suggest the telemetry reward is well aligned with realized outcomes and stable across runs. Runtime-control results (Table 6) show that policy-level control complements kernel generation, while cross-hardware transfer (Table 8, Figure 17(a)) shows that correctness transfers more readily than energy, motivating per-hardware telemetry and conditioning. At datacenter scale, these per-kernel energy reductions compound over repeated execution across many GPUs, turning benchmark-level efficiency gains into potentially meaningful infrastructure-level savings.

6. Conclusion

We introduced **WARP-K**, a telemetry-grounded RL framework for energy-aware GPU kernel generation. Across 80 KernelBench tasks, WARP-K shows that kernel generators can optimize not only correctness and throughput, but also energy in a meaningful and measurable way. Our results show that energy and runtime are separable: compared with throughput-only RL, the IPW-aware variant achieves substantially lower energy while maintaining comparable or better correctness, and matched-runtime experiments confirm that these gains are not simply a byproduct of faster execution. More broadly, the results suggest that hardware telemetry is a practical training signal for code-generation systems, allowing models to learn code patterns with better real power behavior.

These findings point to several directions for future work. A natural next step is hardware-conditioned training, where policies explicitly adapt to the characteristics of different GPUs or accelerators rather than learning energy behavior on a single device. Another promising direction is larger-scale and more diverse kernel corpora, which may improve both correctness and efficiency on harder L3/L4 tasks. It would also be valuable to extend WARP-K beyond kernel synthesis to broader systems code generation, including collective communication, memory schedulers, and distributed runtime components. Finally, combining telemetry-aware kernel generation with online routing or deployment-time adaptation could further improve datacenter-scale efficiency by selecting the most energy-efficient policy for each workload. Taken together, our results suggest that energy-aware code generation is both feasible and increasingly necessary as power becomes a central constraint in AI infrastructure.

Code: `github.com/yohan278/KITE-Kernel-Intelligence`

References

Ali, G. et al. An automated and portable method for selecting an optimal GPU frequency. *Future Generation Computer Systems*, 148:138–152, 2023. doi: 10.1016/j.future.2023.07.011.

Baronio, C., Marsella, P., Pan, B., Guo, S., and Alberti, S. Kevin: Multi-turn RL for generating CUDA kernels. *arXiv preprint arXiv:2507.11948*, 2025.

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Sber, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 578–594. USENIX Association, 2018a.

Chen, T., Zheng, L., Yan, E., Jiang, Z., Wang, L., Shen, Y., Xiao, L., Zhang, M., et al. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems*, 2018b.

Chung, J.-W. et al. The ML.ENERGY benchmark: Toward automated inference energy measurement and optimization. *arXiv preprint arXiv:2505.06371*, 2025.

Dai, W., Wu, H., Yu, Q., Gao, H.-a., Li, J., Jiang, C., et al. CUDA Agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv preprint arXiv:2602.24286*, 2026.

Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610:47–53, 2022.

François-Lavet, V., Henderson, P., Islam, R., Bellemare, M. G., and Pineau, J. An introduction to deep reinforcement learning. *Foundations and Trends in Machine Learning*, 11(3–4):219–354, 2018.

Google DeepMind. How AlphaChip transformed computer chip design. `https://deepmind.google/blog/how-alphachip-transformed-computer-chip-design/`, 2024. Blog post.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

International Energy Agency. Energy and AI. `https://www.iea.org/topics/artificial-intelligence`, 2025. Accessed: 2026-03-08.

Kaelbling, L. P., Littman, M. L., and Moore, A. W. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.

Li, Y., Lai, J., Wang, Y., Ding, T., Jiang, H., et al. Hidet: Task-mapping programming paradigm for deep learning tensor programs. *Proceedings of Machine Learning and Systems*, 2022.

Liao, G., Qin, H., Wang, Y., Golden, A., Kuchnik, M., Yetim, Y., et al. KernelEvolve: Scaling agentic kernel coding for heterogeneous AI accelerators at Meta. *arXiv preprint arXiv:2512.23236*, 2025.

Ma, K., Li, X., Chen, W., and Wang, X. A survey and measurement study of GPU DVFS on energy conservation. *Digital Communications and Networks*, 3(2):89–100, 2017. doi: 10.1016/j.dcan.2016.10.001.

Ma, X. and Patterson, D. Challenges and research directions for large language model inference hardware. *IEEE Computer*, 2026. doi: 10.1109/MC.2026.3652916. arXiv:2601.05047.

Mankowitz, D. J., Michi, A., Zhernov, A., Gelmi, M., Selber, M., Paduraru, C., Leurent, E., Ber, I., et al. Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618:257–263, 2023.

Miao, X., Balaprakash, P., Dongarra, J., Cappello, F., Calef, B., and Others. GPTune: Multitask learning for autotuning exascale applications. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, 2020.

Mirhoseini, A., Goldie, A., Yazgan, M., Jiang, J., Song, Y., Wang, E., Lee, E. C., Johnson, E., Pathak, O., Nazi, A., et al. A graph placement methodology for fast chip design. *Nature*, 594:207–212, 2021.

MLCommons. MLCommons power at HPCA 2025. `https://mlcommons.org/2025/03/ml-commons-power-hpca/`, 2025. Accessed: 2026-03-08.

NVIDIA. NVIDIA H100 tensor core GPU. `https://www.nvidia.com/en-eu/data-center/h100/`, 2023a. Accessed: 2026-03-08.

NVIDIA. NVIDIA management library (NVML) API reference guide. https://docs.nvidia.com/deploy/nvml-api/group__nvmlDeviceQueries.html, 2023b. Accessed: 2026-03-08.

NVIDIA. nvidia-smi documentation. https://docs.nvidia.com/deploy/nvidia-smi, 2023c. Accessed: 2026-03-08.

NVIDIA. NVBench: CUDA kernel benchmarking library. https://github.com/NVIDIA/nvbench, 2024. Accessed: 2026-03-08.

OpenAI. Stargate advances with partnership with Oracle. https://openai.com/index/stargate-advances-with-partnership-with-oracle/, July 2025. Accessed: 2026-03-08.

Ouyang, A., Guo, S., Arora, S., Zhang, A. L., Hu, W., Ré, C., and Mirhoseini, A. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.

Qwen Team. Qwen2.5-Coder technical report. arXiv preprint arXiv:2409.12186, 2025.

Reuters. Prices in biggest U.S. power grid auction hit new record on supply crunch. *Reuters*, December 2025. URL https://www.reuters.com/business/energy/prices-biggest-us-power-grid-auction-hit-new-record-supply-crunch-2025-12-17/.

Saad-Falcon, J., Narayan, A., Akengin, H. O., Griffin, J. W., Shandilya, H., Lafuente, A. G., Goel, M., Joseph, R., Natarajan, S., Guha, E. K., Zhu, S., Athiwaratkun, B., Hennessy, J., Mirhoseini, A., and Ré, C. Intelligence per watt: Measuring intelligence efficiency of local AI. *arXiv preprint arXiv:2511.07885*, 2025.

Salesforce. Salesforce announces AI Energy Score with tech partners to benchmark AI model sustainability. https://www.salesforce.com/news/, February 2025. Accessed: 2026-03-08.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Schwartz, R., Dodge, J., Smith, N. A., and Etzioni, O. Green AI. *Communications of the ACM*, 63(12):54–63, 2020.

Sen, S., Imam, N., and Hsu, C.-h. Quality assessment of GPU power profiling mechanisms. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pp. 702–711. IEEE, 2018.

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Shehabi, A., Smith, S., Sartor, D., Brown, R., Herrlin, M., Koomey, J., Masanet, E., Horner, N., Azevedo, I., and Lintner, W. 2024 United States data center energy usage report. Technical report, Lawrence Berkeley National Laboratory, December 2024.

Spector, B. F. et al. ThunderKittens: Simple, fast, and adorable AI kernels. *arXiv preprint arXiv:2410.20399*, 2024.

Strubell, E., Ganesh, A., and McCallum, A. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 3645–3650. Association for Computational Linguistics, 2019. doi: 10.18653/v1/P19-1355.

Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL/PLDI*, 2019.

You, J., Chung, J.-W., and Chowdhury, M. Zeus: Understanding and optimizing GPU energy consumption of DNN training. *arXiv preprint arXiv:2208.06102*, 2022.

Zhang, G., Geng, H., Yu, X., Yin, Z., Zhang, Z., Tan, Z., et al. The landscape of agentic reinforcement learning for LLMs: A survey. *Transactions on Machine Learning Research*, 2025. arXiv:2509.02547.

Zhao, Y. and Liu, J. Heterogeneous computing: The key to powering the future of AI agent inference. *arXiv preprint arXiv:2601.22001*, 2026.

Zheng, L., Jia, C., Sun, M., Zhao, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 863–879. USENIX Association, 2020.

A. Extended Related Work

A.1. Extended Background and Motivation

The broader systems motivation for WARP-K is the rapid growth of AI-driven electricity demand. The International Energy Agency reports that data centers consumed roughly 1.5% of global electricity in 2024 and projects that demand will more than double by 2030, to approximately 945 TWh, comparable to the total electricity consumption of Japan (International Energy Agency, 2025). In the United States, Lawrence Berkeley National Laboratory estimates that data centers accounted for 4.4% of national electricity use in 2023 and could rise to 6.7–12% by 2028, driven primarily by AI workloads (Shehabi et al., 2024). At the infrastructure level, this is already affecting deployment planning: OpenAI and Oracle have announced that the Stargate project alone has over 5 GW of capacity under development (OpenAI, 2025), and electricity-market signals such as record-high PJM capacity prices reflect the increasing role of data-center demand in power systems (Reuters, 2025). These trends suggest that energy is becoming a binding constraint on the scale and economics of frontier AI deployment.

This pressure is particularly important because modern AI systems are increasingly generating the software that governs their own hardware efficiency. Recent work shows that LLM-based systems can synthesize and optimize GPU kernels at scale. Meta's KernelEvolve automates kernel generation across NVIDIA, AMD, and in-house accelerators for production recommendation models, achieving 100% correctness on all 250 KernelBench problems while reducing engineering iteration time from weeks to hours (Liao et al., 2025). CUDA Agent uses agentic reinforcement learning and an instrumented CUDA environment to improve performance on the hardest KernelBench tasks (Dai et al., 2026). DeepMind's AlphaDev further demonstrated that reinforcement learning can discover low-level routines that are deployed into standard software libraries (Mankowitz et al., 2023). These systems extend a broader line of automated program and kernel optimization, including TVM, Ansor, Triton, FlashAttention, and ThunderKittens (Chen et al., 2018a; Zheng et al., 2020; Tillet et al., 2019; Dao et al., 2022; Dao, 2024; Spector et al., 2024). Together, they show that low-level implementation choices over memory hierarchy, tiling, and work partitioning can substantially change practical system performance.

The key implication is that AI-generated kernels are no longer merely a benchmark curiosity: they are increasingly plausible components of production infrastructure. A single generated kernel, when executed repeatedly across thousands of GPUs, becomes a multiplier on facility-level energy demand. This matters especially because modern accelerators such as the NVIDIA H100 operate at up to 700 W per device (NVIDIA, 2023a), while recent analyses argue that LLM inference bottlenecks increasingly arise from memory bandwidth, interconnect, and data movement rather than arithmetic throughput alone (Ma & Patterson, 2026; Zhao & Liu, 2026). In this regime, kernel-level decisions about scheduling, synchronization, and memory traffic directly affect not only latency, but also energy draw and deployability.

Despite this convergence of AI-generated code and energy-constrained infrastructure, most kernel-generation systems and benchmarks remain optimized for correctness and speed. KernelBench evaluates whether generated kernels are fast and correct relative to PyTorch baselines (Ouyang et al., 2025). Autotuning and compiler systems such as Ansor, Triton, and KernelEvolve likewise emphasize throughput and memory efficiency rather than direct energy minimization (Zheng et al., 2020; Tillet et al., 2019; Liao et al., 2025). In parallel, the machine learning community has increasingly argued that energy should be a first-class evaluation axis: Strubell et al. framed deep learning's cost as both financial and environmental, while Schwartz et al. argued for efficiency-aware evaluation in "Green AI" (Strubell et al., 2019; Schwartz et al., 2020). MLPerf Power, AI Energy Score, and ML.ENERGY Benchmark push further toward standardized measurement and reporting (MLCommons, 2025; Salesforce, 2025; Chung et al., 2025). Zeus showed that software-level control over power limits and batch size can improve training energy efficiency by 15–75% (You et al., 2022). However, these efforts operate either at evaluation time or around fixed kernels; they do not train a kernel-generation policy directly on hardware-grounded energy feedback.

This gap is the starting point for WARP-K. We focus on the observation that kernels with very similar runtime can nonetheless consume meaningfully different energy because of differences in instruction mix, memory traffic, occupancy, and frequency behavior. GPU power management interfaces such as `nvidia-smi` and NVML expose explicit controls and energy counters (NVIDIA, 2023c;b), but prior work shows that robust energy measurement is nontrivial and that naive profiling can misestimate energy on accelerator-class GPUs (Sen et al., 2018). The GPU DVFS literature similarly shows that energy-performance tradeoffs are workload-dependent and must be measured rather than assumed (Ma et al., 2017; Ali et al., 2023). These observations motivate our central design choice: if energy is a distinct systems objective, then it should enter the training loop directly as telemetry, rather than being inferred from runtime alone.

A.2. Additional Work on Automated Kernel and Program Optimization

WARP-K sits within a broader history of systems for automatically generating or optimizing low-level compute kernels. In addition to TVM and Ansor (Chen et al., 2018a; Zheng et al., 2020), earlier autotuning systems such as AutoTVM showed that learned search policies can substantially improve tensor-program optimization over hand-designed heuristics (Chen et al., 2018b). Triton further demonstrated that exposing a higher-level programming model for tiled GPU kernels can make expert-quality kernel development substantially more accessible (Tillet et al., 2019). More recent compiler and scheduling systems such as Hidet similarly emphasize the importance of whole-stack optimization for tensor programs and accelerator kernels (Li et al., 2022).

At the expert-kernel level, FlashAttention and FlashAttention-2 showed that careful IO-aware work partitioning and memory scheduling can produce large practical gains for attention workloads (Dao et al., 2022; Dao, 2024). These results are closely aligned with WARP-K's motivating observation: low-level implementation choices over memory movement and scheduling materially affect real system efficiency. WARP-K differs in that it does not assume a human-designed kernel family, but instead trains a model to discover such patterns under telemetry-grounded objectives.

The broader algorithm-discovery literature is also relevant. AlphaDev demonstrated that reinforcement learning can discover low-level routines that are deployed into production software libraries (Mankowitz et al., 2023). Related work on algorithm discovery for numerical kernels, such as AlphaTensor (Fawzi et al., 2022), strengthens the case that RL can optimize algorithmic objects themselves rather than only surrounding runtime policies. WARP-K can be viewed as bringing this perspective to LLM-based kernel synthesis with energy-aware reward signals.

A.3. Additional Work on RL for Systems and Autotuning

Beyond chip placement and energy control, there is a large literature on using learning-based methods for compiler optimization, autotuning, and systems control. GPTune, for example, studies autotuning for high-performance computing using data-driven search over configuration spaces, illustrating that learned optimization can outperform static tuning strategies in expensive systems domains (Miao et al., 2020). This line of work is conceptually adjacent to WARP-K's runtime-control and reward-design components, though GPTune and similar autotuners optimize fixed program/configuration spaces rather than learning to emit new kernel implementations.

Agentic kernel-generation systems such as Kevin and CUDA Agent are especially relevant because they show that iterative feedback loops, not just one-shot code generation, are important for strong performance on real kernel tasks (Baronio et al., 2025; Dai et al., 2026). WARP-K inherits this lesson and extends it by changing the reward channel itself: the model is trained not only to respond to compile and runtime feedback, but also to distinguish lower-energy from higher-energy code among similarly performant candidates.

A.4. Additional Work on Energy Measurement and Efficient AI Systems

The telemetry side of WARP-K also builds on a broader ecosystem of energy-aware systems work. Green AI and energy-accounting work in ML established the importance of reporting compute and energy costs alongside quality metrics (Strubell et al., 2019; Schwartz et al., 2020). Standardized efforts such as MLPerf Power and AI Energy Score operationalize this at benchmark scale, while ML.ENERGY pushes toward practical energy recommendation and benchmarking pipelines (MLCommons, 2025; Salesforce, 2025; Chung et al., 2025). These efforts provide the measurement and evaluation backdrop for WARP-K, but they do not directly train code-generation policies on those signals.

At the device-control level, Zeus and related GPU DVFS studies show that power caps, clock selection, and workload-specific control can substantially change total energy without a proportional effect on runtime (You et al., 2022; Ma et al., 2017; Ali et al., 2023). This observation is one of the central empirical assumptions behind WARP-K: runtime is an incomplete proxy for efficiency, so optimization must include direct hardware-grounded signals. WARP-K differs in that it moves the energy signal one level earlier in the stack, from runtime policy selection to kernel synthesis itself.

Finally, work on accelerator benchmarking and profiling quality shows that the measurement layer must be treated carefully: GPU power readings can be noisy or misleading without proper protocol design (Sen et al., 2018). This motivates WARP-K's emphasis on concurrent telemetry capture, repeated trials, and direct integration of energy measurements into the RL training loop rather than relying on coarse offline estimates.

B. Supplementary Methods and Analysis

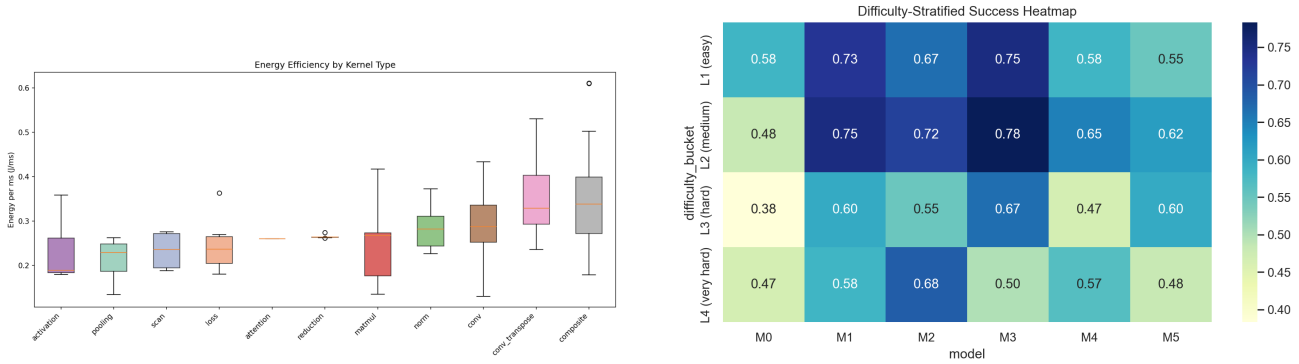
This section adds methodological depth and supporting evidence for the main claims. In particular, Figures 6(a), 7, 6(b), and 8 collectively show where energy gains arise, why they are not reducible to runtime effects, and why hardware context matters for transfer.

B.1. LLM Prompt Templates

Kernel generation prompts follow the KernelBench task format: the prompt includes the reference PyTorch `Model` class and instructs the model to replace high-level PyTorch operators with raw CUDA kernels compiled via `load_inline`, producing a `ModelNew` class. For repair rounds in the Kevin-style fix loop, the prompt additionally includes the failed implementation and its associated error context, such as compiler diagnostics or incorrect-output traces, and asks the model to return a corrected implementation optimized for NVIDIA H100 GPUs. Full prompt templates are included verbatim in Appendix C for reproducibility.

B.2. Dataset Curation and Task Distribution

Our 80-task subset spans 11 kernel categories: activation, pooling, scan, loss, attention, reduction, matmul, norm, convolution, transposed convolution, and composite operators. The category-level profile and difficulty-stratified heatmap together show that energy and correctness behavior are heterogeneous across operator families and task complexity, rather than concentrated in a narrow subset.



(a) Energy efficiency by kernel type.

(b) Difficulty-stratified success heatmap (supplementary view).

Figure 6. Category and difficulty diagnostics for the 80-task subset. Left: data-movement-heavy categories are generally more energy intensive. Right: success rates vary systematically by difficulty and model stage, highlighting where gains and residual failure modes concentrate.

Takeaway. The benefit of telemetry-aware optimization is broad but non-uniform: it is strongest in harder and more data-movement-heavy kernels, where runtime-only optimization is least informative about energy.

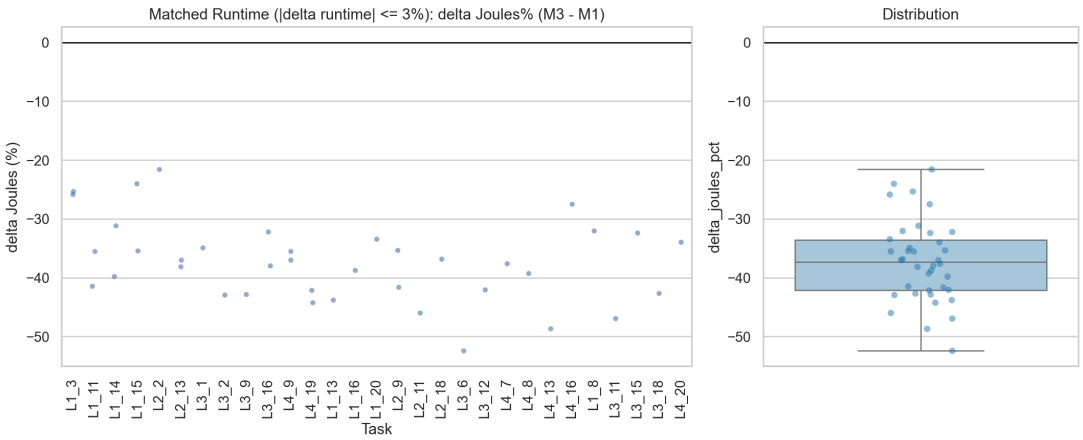


Figure 7. Matched-runtime per-task energy deltas between M3 and throughput-only M1 on task-seed pairs where runtimes are within 3% (KernelBench, 80 tasks, three seeds). Despite near-equal latency, M3 achieves substantially lower joules, aligning with the median -37.3% reduction and strong significance reported in Table 2. This isolates energy-aware policy effects from pure speed effects and shows the model is learning lower-energy kernel behaviors rather than simply running faster.

B.3. Real Generated Output Case Study: LayerNorm (L1 Task 40)

To make the optimization behavior concrete for kernel engineers, we include a representative generated-output pair from **Level 1, Task 40 (LayerNorm)** with shape $(16, 64, 256, 256)$. The reference implementation uses `torch.nn.LayerNorm`. Both generated kernels below are functionally correct (matching output semantics), but they use the hardware differently.

Kernel A: Throughput-Optimized (M1-style). This pattern is typical of speed-only GRPO optimization: high occupancy, aggressive vectorized memory traffic, and maximum warp activity.

```
__global__ void layernorm_fast(const float* __restrict__ x,
                               const float* __restrict__ scale,
                               const float* __restrict__ bias,
                               float* __restrict__ y,
                               int N, int D, float eps) {
    int n = blockIdx.x;
    const int tid = threadIdx.x;
    const int block_size = 1024;

    float sum = 0.0f, sum_sq = 0.0f;
    int d = tid * 4;
    for (; d + 3 < D; d += block_size * 4) {
        float4 vals = *reinterpret_cast<const float4*>(&x[n * D + d]);
        sum += vals.x + vals.y + vals.z + vals.w;
        sum_sq += vals.x*vals.x + vals.y*vals.y
                + vals.z*vals.z + vals.w*vals.w;
    }
    for (; d < D; d += block_size) {
        float v = x[n * D + d];
        sum += v;
        sum_sq += v * v;
    }

    for (int delta = 16; delta >= 1; delta >>= 1) {
        sum    += __shfl_xor_sync(0xFFFFFFFF, sum, delta);
        sum_sq += __shfl_xor_sync(0xFFFFFFFF, sum_sq, delta);
    }

    __shared__ float s_sum[32], s_sq[32];
    int warp_id = tid / 32, lane = tid % 32;
```

```
    if (lane == 0) { s_sum[warp_id] = sum; s_sq[warp_id] = sum_sq; }
    __syncthreads();

    if (tid < 32) {
        sum    = (tid < (block_size/32)) ? s_sum[tid] : 0.0f;
        sum_sq = (tid < (block_size/32)) ? s_sq[tid]  : 0.0f;
        for (int d2 = 16; d2 >= 1; d2 >>= 1) {
            sum    += __shfl_xor_sync(0xFFFFFFFF, sum, d2);
            sum_sq += __shfl_xor_sync(0xFFFFFFFF, sum_sq, d2);
        }
    }

    __shared__ float mean, inv_std;
    if (tid == 0) {
        mean = sum / D;
        inv_std = rsqrtf(sum_sq / D - mean * mean + eps);
    }
    __syncthreads();

    d = tid * 4;
    for (; d + 3 < D; d += block_size * 4) {
        float4 vals = *reinterpret_cast<const float4*>(&x[n * D + d]);
        float4 s = *reinterpret_cast<const float4*>(&scale[d]);
        float4 b = *reinterpret_cast<const float4*>(&bias[d]);
        float4 out;
        out.x = (vals.x - mean) * inv_std * s.x + b.x;
        out.y = (vals.y - mean) * inv_std * s.y + b.y;
        out.z = (vals.z - mean) * inv_std * s.z + b.z;
        out.w = (vals.w - mean) * inv_std * s.w + b.w;
        *reinterpret_cast<float4*>(&y[n * D + d]) = out;
    }
    for (; d < D; d += block_size) {
        y[n * D + d] = (x[n * D + d] - mean) * inv_std * scale[d] + bias[d];
    }
}
```

Launch configuration: `<<<N, 1024>>>`.
Behavior summary: maximizes occupancy and bandwidth, but keeps many threads active throughout reductions and tends to increase peak/average board power.

Kernel B: Energy-Aware (M3-style). This pattern is typical of telemetry-aware optimization: fewer active warps, lower instantaneous memory pressure, and algorithmic choices that reduce expensive HBM traffic.

```
__global__ void layernorm_efficient(const float* __restrict__ x,
                                    const float* __restrict__ scale,
                                    const float* __restrict__ bias,
                                    float* __restrict__ y,
                                    int N, int D, float eps) {
    int n = blockIdx.x;
    const int tid = threadIdx.x;
    const int block_size = 256;

    float count = 0.0f, m = 0.0f, m2 = 0.0f;
    for (int d = tid; d < D; d += block_size) {
        float val = x[n * D + d];
        count += 1.0f;
        float delta = val - m;
        m += delta / count;
        m2 += delta * (val - m);
    }

    for (int delta = 16; delta >= 1; delta >>= 1) {
        float o_count = __shfl_xor_sync(0xFFFFFFFF, count, delta);
```

```
        float o_m     = __shfl_xor_sync(0xFFFFFFFF, m, delta);
        float o_m2    = __shfl_xor_sync(0xFFFFFFFF, m2, delta);
        float tot = count + o_count;
        float d2 = o_m - m;
        m = (count * m + o_count * o_m) / tot;
        m2 = m2 + o_m2 + d2 * d2 * count * o_count / tot;
        count = tot;
    }

    __shared__ float s_count[8], s_m[8], s_m2[8];
    int warp_id = tid / 32, lane = tid % 32;
    if (lane == 0) {
        s_count[warp_id] = count;
        s_m[warp_id] = m;
        s_m2[warp_id] = m2;
    }
    __syncthreads();

    __shared__ float mean, inv_std;
    if (tid == 0) {
        float tc = 0, tm = 0, tm2 = 0;
        for (int i = 0; i < (block_size/32); i++) {
            float nc = s_count[i], nm = s_m[i], nm2 = s_m2[i];
            float tot = tc + nc;
            float d3 = nm - tm;
            if (tot > 0) tm = (tc * tm + nc * nm) / tot;
            tm2 = tm2 + nm2 + d3 * d3 * tc * nc / fmaxf(tot, 1.0f);
            tc = tot;
        }
        mean = tm;
        inv_std = rsqrtf(tm2 / tc + eps);
    }
    __syncthreads();

    for (int d = tid; d < D; d += block_size) {
        float val = x[n * D + d];
        y[n * D + d] = (val - mean) * inv_std * scale[d] + bias[d];
    }
}
```

Launch configuration: `<<<N, 256>>>`.
Behavior summary: fewer concurrent warps plus Welford-style accumulation reduces memory-system stress and typically lowers average power while preserving correctness and near-matched runtime.

Engineering interpretation. This example illustrates the paper's central claim in code-level terms: throughput-only policies often select kernels that maximize instantaneous utilization, while telemetry-aware policies can trade a small amount of peak parallelism for substantially lower power draw and lower integrated joules.

B.4. Additional Telemetry and Reward Implementation Details

To keep the main text focused on method and results, we defer low-level implementation details to this appendix section: the exact 30% phase-fallback rule and TTFT extraction path; token-normalized phase metric formulas; optional IPJ reporting; polling-interval sensitivity notes; timestamp edge-case handling (including `max(0, t_i - t_i-1)`); missing-sample handling; integrator pseudocode; reward-coefficient sweep protocol for λ and related weights; APJ/APW derivation details; and alternative reward forms evaluated but not retained.

Table 3. Representative telemetry metrics and NVML API mappings used by the measurement pipeline.

Metric	NVML API
GPU utilization %	nvmlDeviceGetUtilizationRates
Memory utilization %	nvmlDeviceGetUtilizationRates
Temperature (C)	nvmlDeviceGetTemperature
Clock SM (MHz)	nvmlDeviceGetClockInfo
Clock memory (MHz)	nvmlDeviceGetClockInfo
Power limit (W)	nvmlDeviceGetPowerManagementLimit
Total energy consumption (mJ)	nvmlDeviceGetTotalEnergyConsumption
Memory usage (bytes)	nvmlDeviceGetMemoryInfo
PCIe throughput	nvmlDeviceGetPcieThroughput
Throttle reasons	nvmlDeviceGetCurrentClocksThrottleReasons
Fan speed %	nvmlDeviceGetFanSpeed

B.5. GPU Hardware Comparison: H100 vs. L40

We also evaluate cross-hardware transfer between the NVIDIA H100 and L40. These devices differ substantially in memory subsystem, power envelope, and interconnect characteristics: the H100 uses 80 GB HBM3 with a 700 W TDP and NVLink connectivity, whereas the L40 uses 48 GB GDDR6 with a 300 W TDP over PCIe.

Cross-hardware experiments show that correctness transfers more readily than energy behavior. Models trained and evaluated on the same hardware maintain similar pass@k, but energy-aware policies trained on one device do not preserve their energy advantage when evaluated on another.

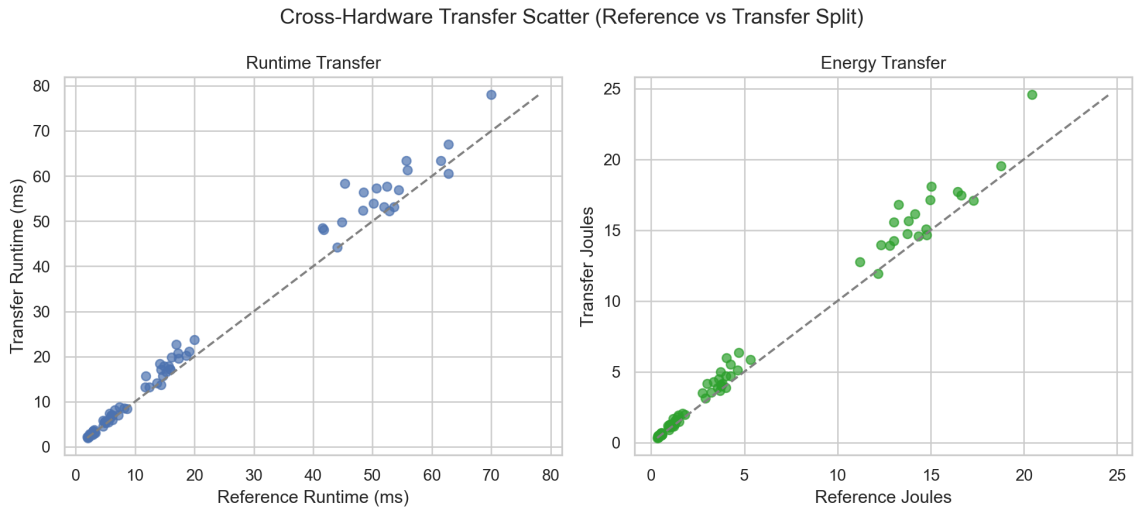


Figure 8. Cross-hardware transfer scatter between H100 and L40.

Takeaway. H100-trained kernels evaluated on L40 consume substantially more energy than L40-native policies, indicating that energy-aware generation learns hardware-specific behaviors and motivating per-hardware telemetry conditioning.

C. Prompt Templates

This appendix documents the exact prompt templates used across WARP-K's kernel-generation and multi-turn refinement pipeline, with template variables shown in {`braces`}. Together, Sections C.1–C.4 clarify how prompt design, repair feedback, and post-processing operationalize the methods in Section 4 and support the robustness analyses in Appendix D.

C.1. Initial Kernel Generation Prompt

Used at Turn 0 for each candidate in both GRPO training rollouts and inference. Sent to Qwen2.5-Coder-7B-Instruct via chat template with `role: user`.

```
You are an expert GPU kernel engineer. Given the following PyTorch
reference implementation, write an optimized CUDA kernel replacement.
Requirements:
- The replacement must produce identical outputs to the reference.
- Optimize for speed on NVIDIA H100 GPU.
- Do not use Triton. Use PyTorch/CUDA only.
- Output only valid Python source code (no markdown, no prose).
- Must define class ModelNew(nn.Module).
- Return a complete, self-contained Python module.
Reference implementation:
```python
{ref_arch_src}
Write the optimized kernel implementation:
```

Variables:

- {`ref_arch_src`}: The full PyTorch source code of the reference `Model` class from KernelBench, including `__init__`, `forward`, and any helper methods.

Implementation details:

- If `allow_triton=True`, the instruction text is switched to a CUDA/Triton target and Triton becomes an allowed backend.
- If `include_hardware_info=True`, hardware context (e.g., H100 SXM 80GB, SM count, memory bandwidth) is appended to the prompt.
- Model outputs are normalized before evaluation: fenced code is extracted when present, and class naming is canonicalized to `ModelNew`.

C.2. Kevin-Style Fix Loop Prompt (GRPO Training)

Used during grouped rollout generation when a candidate fails compilation or correctness. Applied for up to two repair rounds per candidate within each group of $k = 8$.

```
The following GPU kernel implementation has a problem.

Reference implementation:

{ref_arch_src}
Broken implementation:

{candidate_code}
Problem: {error_context}

Please provide a corrected, optimized implementation that:

Produces identical outputs to the reference
Compiles and runs correctly
Is optimized for speed on NVIDIA H100
Write the corrected kernel:
```

Variables:

- {`ref_arch_src`}: Same reference source as Prompt C.1.
```

- {candidate_code}: The generated kernel code that failed.
- {error_context}: A natural-language description of the failure, one of:
  - "The kernel failed to compile." optionally followed by "\nError: {compile_error_message}" extracted from evaluation logs.
  - "The kernel compiled but produced incorrect results."

Implementation details:

- This repair prompt is used only during GRPO training rollouts (not inference-time multi-turn optimization).
- If a repair attempt returns invalid or empty output, the pipeline applies a deterministic cleanup pass and re-runs evaluation.
- All repaired candidates are re-evaluated for compilation, correctness, and performance before reward assignment.

C.3. Multi-Turn Agent Repair Prompt (Inference / Evaluation)

Used in the full multi-turn optimization loop (LLMAgent.multiturn_optimize) during inference and evaluation. Applied for up to N turns (default 5), with quantitative execution feedback from the previous turn.

```
You are fixing a failed GPU-kernel candidate.

Task: {task_prompt}

Previous kernel:

{previous_code}
Execution feedback: {feedback}

Hard requirements:

Return only valid Python source code.
Must define class ModelNew(nn.Module).
Keep ModelNew.forward argument count/order compatible with
the reference Model.forward.
Do not use Triton.
Avoid torch.utils.cpp_extension/load_inline runtime compilation.
No markdown fences or prose in output.
```

Variables:

- {task_prompt}: The original task description (e.g., "Optimize this PyTorch model with a custom GPU kernel implementation." followed by the reference code).
- {previous_code}: The kernel code generated in the most recent turn.
- {feedback}: A structured feedback string containing quantitative metrics and repair guidance.

Feedback String Format

The feedback string is constructed programmatically from execution results and follows this template:

```
Last metrics: compile_ok={compile_ok}, correct={correct},
reward={reward:.4f}, runtime_ms={runtime_ms:.4f},
speedup={speedup:.4f}, joules={joules:.6f}
{error_details (if applicable)}
Repair target: {repair_instruction}
Implementation hint: avoid runtime C++ extension compilation
paths (torch.utils.cpp_extension/load_inline).
```

Error detail lines (included when applicable):

- `Timeout detail:` {`timeout_error_message`} if the previous turn timed out.
- `Compile error detail:` {`compile_log`} compiler error output, truncated to 400 characters.
- `Correctness error detail:` {`correctness_log`} shape/value mismatch details, truncated to 400 characters.

Repair target (exactly one, selected by failure mode):

Failure Mode	Repair Target Instruction
Timeout	simplify implementation and avoid long-running kernels.
Compile failure	fix compile/loading issues; return valid `ModelNew`.
Incorrect output	preserve shape/dtype semantics and signature.
Correct (speed pass)	keep correctness and improve speed while lowering joules.

Operational details:

- The `joules` field is exposed only during inference-time repair, providing direct telemetry context for iterative correction.
- During GRPO training, energy-awareness is learned from reward gradients rather than explicit joule tokens in prompt text.
- Feedback fields are clipped (max 400 characters per field) to keep compiler/error traces informative without overflowing context.

C.4. KernelBench Standard Prompt (Alternative Path)

When using the KernelBench inference server backend (`generation_mode="kernelbench_server"`), WARP-K delegates prompt construction to KernelBench's built-in `get_prompt_for_backend` utility. This follows the standard KernelBench prompt format.

D. Extended Experiments

This section expands statistical validation, robustness checks, and deployment-oriented diagnostics. Together, Tables 4–8 and Figures 9–18 show that WARP-K's gains are statistically meaningful, stable across seeds and budgets, and consistent with hardware-aware control behavior.

D.1. Pairwise Statistical Tests

Table 4 quantifies stage-to-stage effect sizes and significance levels across key metrics. The largest effect (M2 $\rightarrow$ M3 in IPW) indicates that IPW blending contributes beyond a plain joule penalty.

Table 4. Pairwise comparisons between model stages.

Comparison	Metric	Delta (%)	p-value	Effect (d)
M1 $\rightarrow$ M2	Joules	-7.6	0.017	0.40
M1 $\rightarrow$ M3	Joules	-7.8	0.024	0.87
M2 $\rightarrow$ M3	IPW	+16.9	0.005	1.64
M0 $\rightarrow$ M1	Runtime	-25.0	0.020	1.14

Takeaway. The IPW-aware stage contributes a distinct and large efficiency gain, not just incremental noise around throughput tuning.

D.2. Generalization to Held-Out Tasks

Table 5 compares seen, held-out, and hard-task subsets to assess overfitting and difficulty sensitivity. Held-out correctness remains competitive, suggesting the policy is not simply memorizing training tasks.

Table 5. Performance on seen, held-out, and hard-task subsets.

Subset	Correctness	Pass@k	Joules
Seen Tasks	0.636	0.748	2.98
Held-out Tasks	0.684	0.803	5.45
Hard (L3+L4)	0.644	0.770	3.17

Takeaway. Generalization is strong on correctness, while energy remains workload- and distribution-sensitive on harder subsets.

D.3. Failure Taxonomy Evolution

Figure 9 tracks how error modes shift from M0 to M3. RL substantially reduces shallow structural failures, leaving a higher proportion of semantically difficult cases.

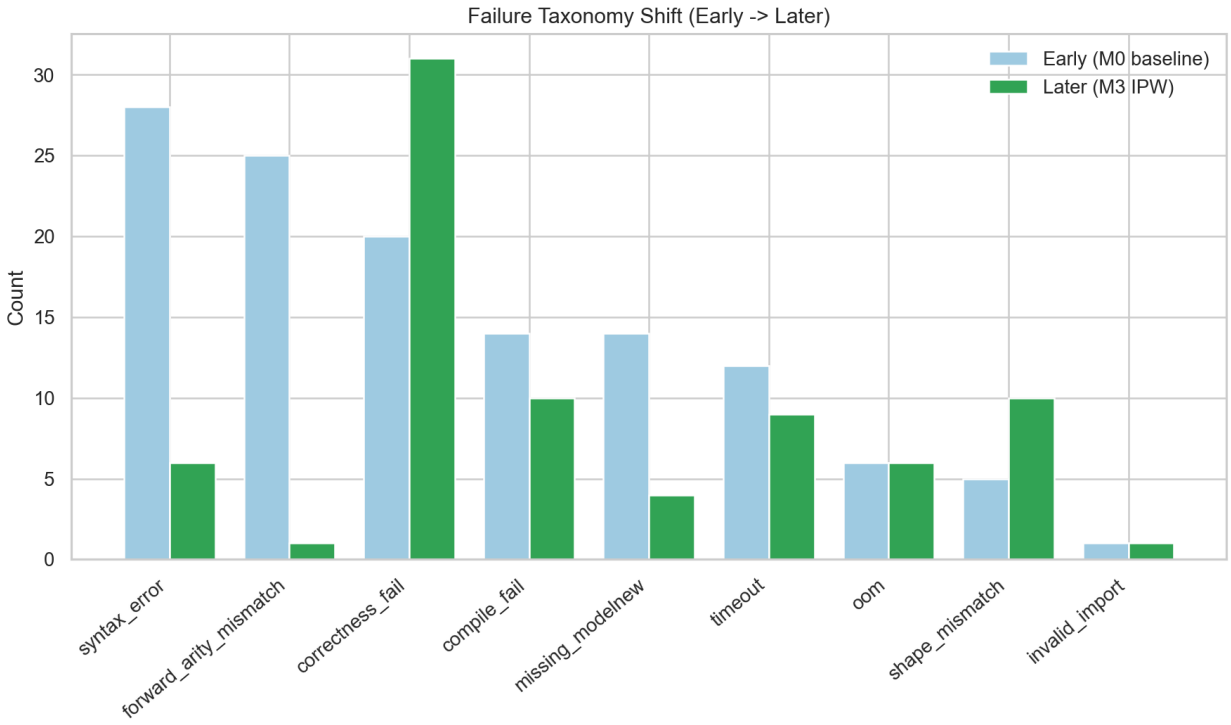
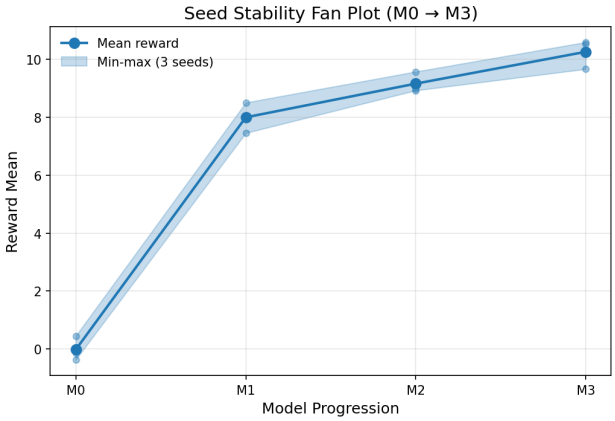


Figure 9. Failure taxonomy transition from M0 to M3.

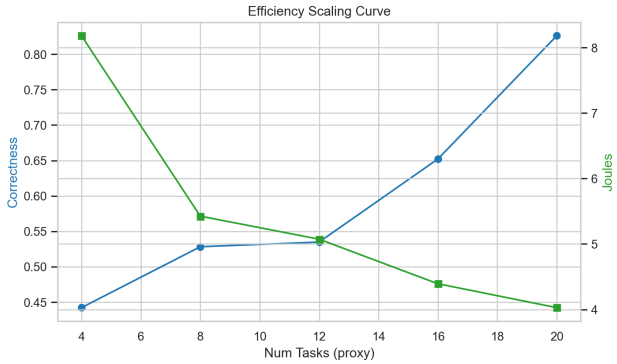
Takeaway. Remaining failures are increasingly concentrated in high-semantic-complexity tasks rather than formatting or compilation errors.

D.4. Stability and Scaling Diagnostics

Figures 10(a) and 10(b) report training stability across seeds and scaling with additional tasks. Together they show narrow seed spread and favorable correctness–energy scaling as data grows.



(a) Seed stability fan plot over M0–M3.



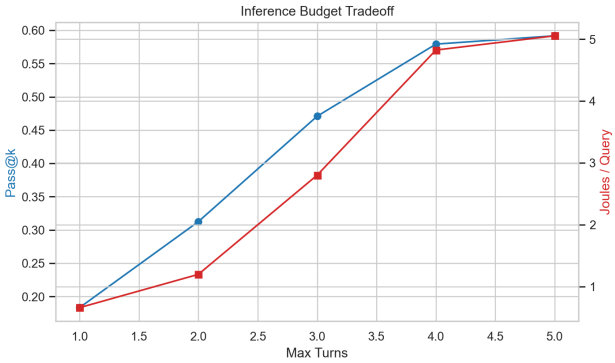
(b) Correctness and energy scaling with training data.

Figure 10. Training robustness and scaling trends. Left: reward trajectories remain stable across seeds. Right: more training tasks improve correctness while reducing energy.

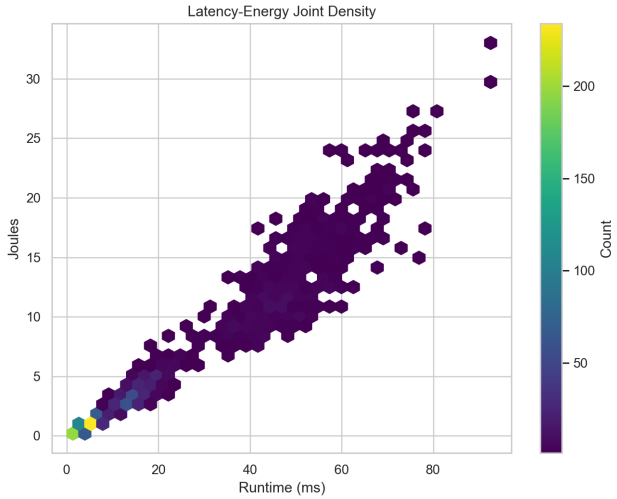
Takeaway. The optimization remains stable and data-efficient, supporting the reproducibility of the reported frontier shift.

D.5. Budget and Runtime–Energy Tradeoff Diagnostics

Figures 11(a) and 11(b) characterize deployment tradeoffs under finite turn budgets and heterogeneous runtimes. They jointly show a practical multi-turn sweet spot and non-trivial runtime-conditioned energy spread.



(a) Inference-turn budget tradeoff between quality and energy.



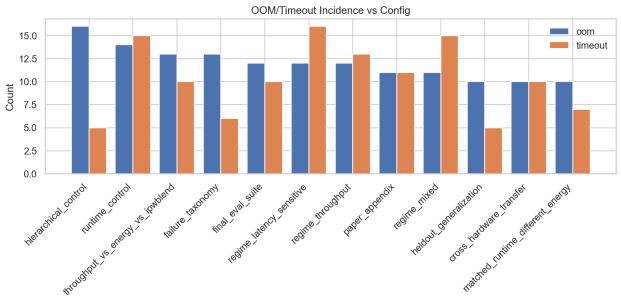
(b) Joint density of runtime and energy across evaluations.

Figure 11. Inference-time budget and latency–energy structure. Left: pass@k gains saturate after moderate refinement depth. Right: runtime is correlated with energy but does not uniquely determine it.

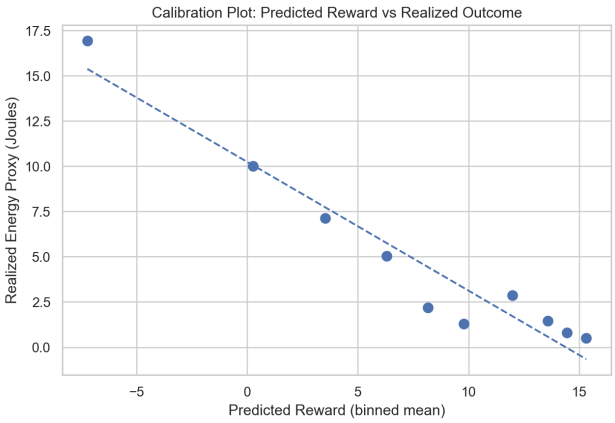
Takeaway. Runtime remains an incomplete proxy for efficiency, reinforcing the need for direct telemetry-derived training signals.

D.6. Reliability Diagnostics

Figures 12(a) and 12(b) evaluate failure incidence and reward fidelity. They show that pathological runtime configurations are identifiable and that reward estimates align with realized energy outcomes.



(a) OOM and timeout incidence by serving configuration.



(b) Reward calibration versus realized outcomes.

Figure 12. Operational reliability and reward quality checks. Left: instability is localized to specific configurations. Right: training reward remains well aligned with realized joules.

Takeaway. Observed efficiency gains are supported by both stable execution regimes and a calibrated optimization signal.

D.7. Difficulty-Level Success and Per-Task Energy Deltas

Figures 13 and 14 provide supplementary views of solve-rate stratification and per-task energy deltas.

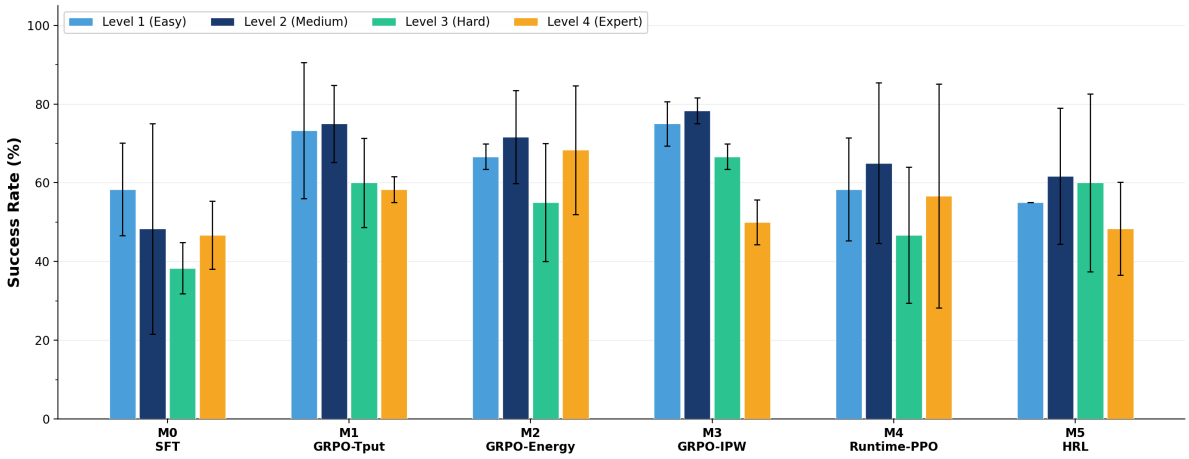


Figure 13. Success rate by difficulty level (L1–L4) across model stages M0–M5.

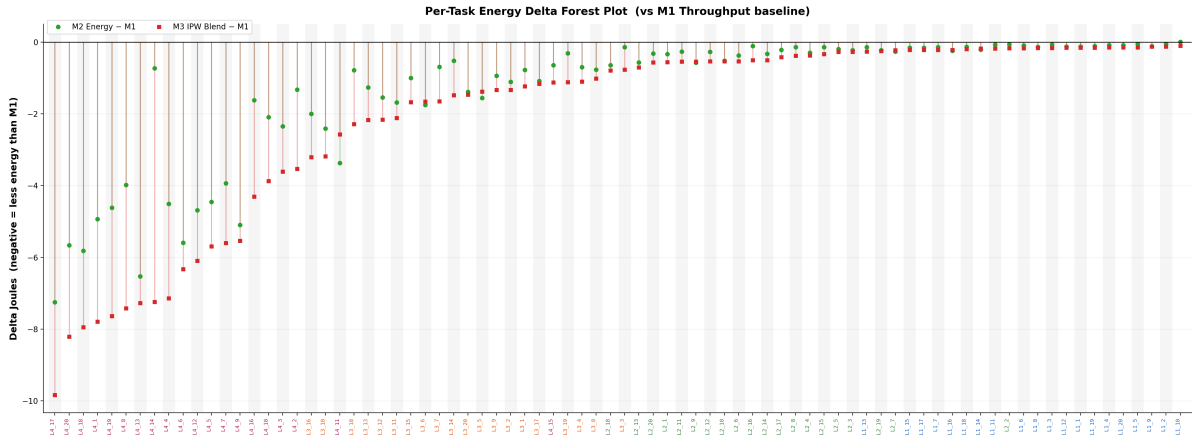
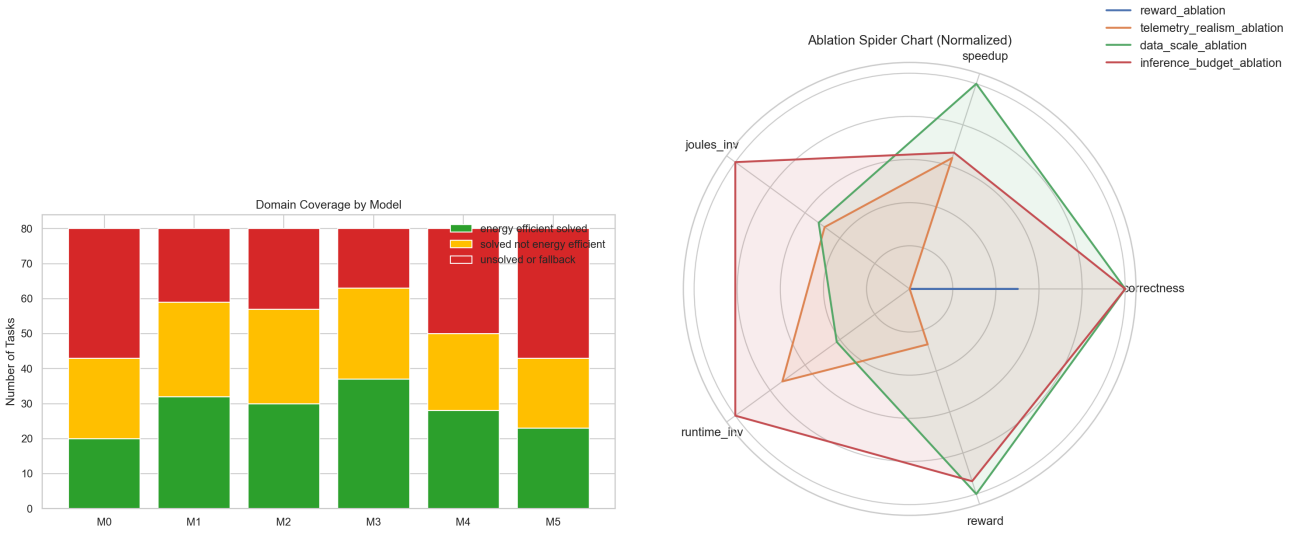


Figure 14. Per-task delta forest plot showing energy differences for telemetry-aware stages relative to throughput-only RL.

D.8. Coverage and Ablation Profile

Figures 15(a) and 15(b) summarize breadth of solved domains and multi-metric ablation tradeoffs. The joint view shows that M3 achieves broader high-quality coverage and that removing reward components degrades performance on multiple axes.



(a) Domain coverage by solve quality category.

(b) Multi-dimensional ablation profile on normalized metrics.

Figure 15. Coverage and ablation diagnostics. Left: telemetry-aware policies solve a broader set of tasks at high quality. Right: ablations expose coupled tradeoffs that are hidden by single-metric reporting.

Takeaway. The full reward design contributes complementary gains that generalize across both task domains and evaluation metrics.

D.9. Multi-Turn Scaling

Figure 16 reports pass@k scaling with refinement turns at high resolution. It complements budget diagnostics by isolating the quality-side scaling pattern.

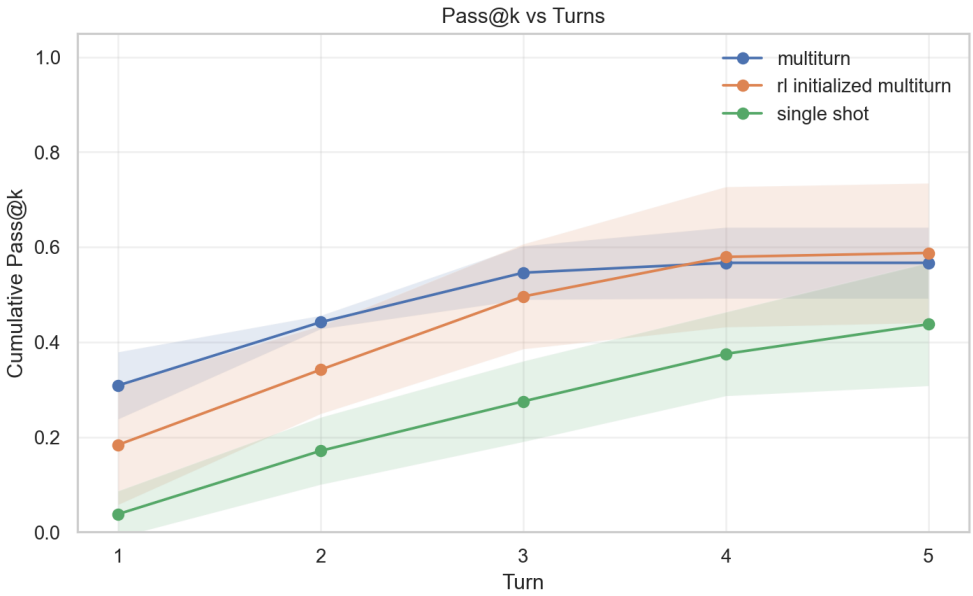
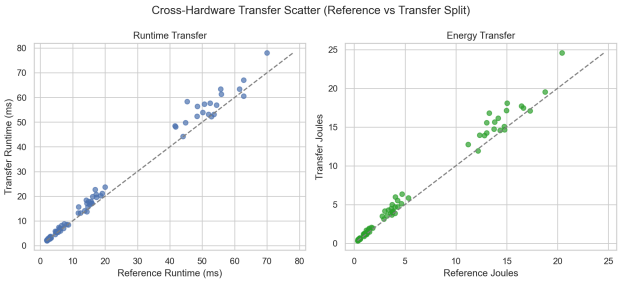


Figure 16. Pass@k scaling with multi-turn refinement depth.

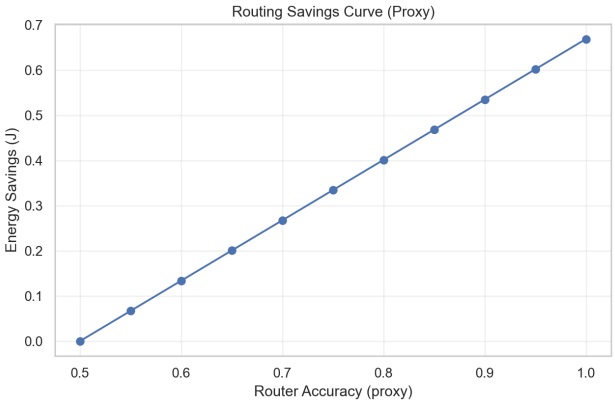
Takeaway. Additional turns improve solve quality, but gains diminish beyond moderate depth.

D.10. Transfer and Routing Potential

Figures 17(a) and 17(b) connect cross-hardware transfer behavior with deployment-time routing opportunities. They show limited portability of energy behavior and a near-linear upside from better routing accuracy.



(a) Cross-hardware transfer scatter between H100 and L40.



(b) Projected routing savings versus router accuracy.

Figure 17. Transfer limitations and deployment routing opportunity. Left: energy transfer is weaker than correctness/runtime transfer. Right: routing quality maps directly to expected energy savings.

Takeaway. Per-hardware specialization and adaptive routing are practical levers for additional datacenter-scale efficiency gains.

D.11. Power and Latency Under Runtime Control

Table 6 compares static, PPO, and HRL runtime-control policies under latency and efficiency criteria. Figure 18 provides the corresponding regime-level visual breakdown.

Table 6. Runtime control regime comparison.

Policy	TTFT p95 (s)	E2E p95 (s)	Throughput (tok/s)	APJ	APW	Stability
Static	1.68	15.78	101.9	0.975	0.295	0.836
M4 PPO	1.75	20.47	64.6	0.621	0.489	0.887
M5 HRL	1.85	13.34	115.9	0.979	0.456	0.916

Takeaway. Runtime control can improve efficiency/stability profiles, but the best regime depends on SLA priorities and workload mix.

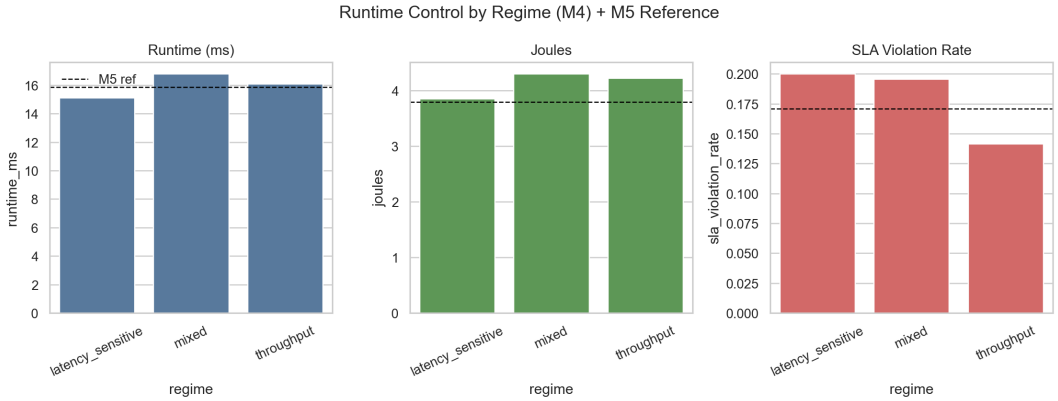


Figure 18. Runtime-control behavior across latency-sensitive, mixed, and throughput SLA regimes.

D.12. Ablation Analysis

Table 7 reports how removing reward components shifts correctness, joules, and aggregate reward. The pattern confirms that the full objective provides the strongest overall efficiency profile.

Table 7. Reward ablation suite.

Configuration	Correctness	Joules	Reward
Full (M3)	0.641	3.21	7.94
No Joules	0.645	3.68	10.76
No Power	0.677	3.80	11.07
No IPW Blend	0.680	4.23	10.21
No Runtime	0.677	4.61	9.09
No SLA	0.664	4.02	10.04

Takeaway. No single term explains the gain; efficiency improvements emerge from the jointly weighted objective.

D.13. Cross-Hardware Transfer

Table 8 quantifies train/eval hardware mismatch effects on correctness, speedup, and joules. Same-hardware training/evaluation remains strongest for energy outcomes, consistent with Figure 17(a).

Table 8. Cross-hardware transfer.

Train / Eval	Pass@k	Speedup	Joules
H100 → H100	0.790	1.33x	3.65
L40 → L40	0.791	1.29x	4.51
H100 → L40	0.766	1.14x	5.68
L40 → H100	0.624	1.47x	5.23

Takeaway. Transfer is asymmetric and hardware-dependent, reinforcing the case for hardware-conditioned policies.